Document Information	
Filename	TP_F101U-00028_User_Guide_Software_Loading_Manager
Status	Release
Revision Date	Jun 10, 2024
Version	19.00
Document ID	TP_F101U-00028

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction 5
1.1 Notational Conventions 8
2 Setup 8
2.1 Tools/Equipment Needed 8
2.2 Setup Instructions 8
2.3 Setup Troubleshooting 9
3 Features/Components 9
3.1 Software Loading Update Package 9
3.1.1 Overview 9
3.1.1.1 Platform Content Overview 9
3.1.1.2 Product Expectations 10
3.1.1.3 Extending Platform Functionality 10
3.1.2 Setup 10
3.1.2.1 Tools/Equipment Needed 10
3.1.2.2 Setup Instructions 10
3.1.2.3 Setup Troubleshooting 10
3.1.3 Usage 11
3.1.3.1 Basic Instructions 11
3.1.3.2 Detailed Instructions 11
3.1.3.3 Testing/Error Reporting 12
3.2 Software Loading Persistency 12
3.2.1 Overview 12
3.2.1.1 Platform Content Overview 12
3.2.1.2 Product Expectations 12
3.2.1.3 Extending Platform Functionality 12
3.2.2 Setup 12
3.2.2.1 Tools/Equipment Needed 12
3.2.2.2 Setup Instructions 12
3.2.2.3 Setup Troubleshooting 12
3.2.3 Usage 12
3.2.3.1 Basic Instructions 12
3.2.3.2 Detailed Instructions 13
3.2.3.3 Testing/Error Reporting 14
3.3 Software Loading Validation 14
3.3.1 Overview 14
3.3.1.1 Platform Content Overview 14
3.3.1.2 Product Expectations 14
3.3.1.3 Extending Platform Functionality 15
3.3.2 Setup 15

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.3.2.1 Tools/Equipment Needed 15
3.3.2.2 Setup Instructions 15
3.3.2.3 Setup Troubleshooting 15
3.3.3 Usage 15
3.3.3.1 Basic Instructions 15
3.3.3.2 Detailed Instructions 16
3.3.3.3 Testing/Error Reporting 17
3.4 Software Loading Update 17
3.4.1 Overview 17
3.4.1.1 Platform Content Overview 17
3.4.1.2 Product Expectations 17
3.4.1.3 Extending Platform Functionality 18
3.4.2 Setup 18
3.4.2.1 Tools/Equipment Needed 18
3.4.2.2 Setup Instructions 19
3.4.2.3 Setup Troubleshooting 19
3.4.3 Usage 19
3.4.3.1 Basic Instructions 19
3.4.3.2 Detailed Instructions 22
3.4.3.3 Update workflows 27
3.4.3.4 Testing/Error Reporting 27
3.5 Software Loading Cancel 27
3.5.1 Overview 27
3.5.1.1 Platform Content Overview 27
3.5.1.2 Product Expectations 27
3.5.1.3 Extending Platform Functionality 28
3.5.2 Setup 28
3.5.2.1 Tools/Equipment Needed 28
3.5.2.2 Setup Instructions 28
3.5.2.3 Setup Troubleshooting 28
3.5.3 Usage 28
3.5.3.1 Basic Instructions 28
3.5.3.2 Detailed Instructions 28
3.5.3.3 Testing/Error Reporting 30
3.6 **Software Loading Information Retrieval 31**
3.6.1 Overview 31
3.6.1.1 Platform content Overview 31
3.6.1.2 Product Expectations 31
3.6.1.3 Extending Platform Functionality 31
3.6.2 Setup 31
3.6.2.1 Tools/Equipment Needed 31
3.6.2.2 Setup Instruction 31
3.6.2.3 Setup Troubleshooting 31

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.3 Usage 32
3.6.3.1 Basic Instructions 32
3.6.3.2 Detailed Instructions 32
3.6.3.3 Testing/Error Reporting 33
4 Known Issues 33
5 FAQs 33
6 Troubleshooting/Tips and Tricks 33
7 References 33
7.1 Abbreviations/Acronyms/Definitions 33
7.2 Platform Documentation 33
7.3 Website links 33
8 Revision History 34

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information for Software Loading Manager. The feature is part of the Telematics Platform Software Loading framework that includes processes running on the applications processor within the System on Chip of the Network Access Device of a Telematics Control Unit. The Software Loading framework is implemented and maintained as part of the middleware provided by the Telematics Platform. This guide focuses on providing an overview on how the Project Application should use the Software Loading framework.

The Software Loading framework is made of the following elements:

- A library and its application programming interface
- A Software Loading Manager
- Update Agents

Thus the Software Loading framework along with 3rd party packages and other components of the TCU allows to perform Over-The-Air reprogramming on some of its elements according to Open Mobile Alliance standards such as Firmware Update Management Object or Software Component Management Object; both relying on the Device Management specification.

The TP Software Loading framework also has a key role when most elements of the TCU are programmed over a vehicle bus, typically CAN protocol.

Since the TP Software Loading framework is formed by several services which rely on an inter-process communication mechanism which is DBUS method and sockets to send messages and data between its different members. This mechanism is abstracted from the Project Application by means of the TP Software Loading library and it's API which is provided for C programming language.

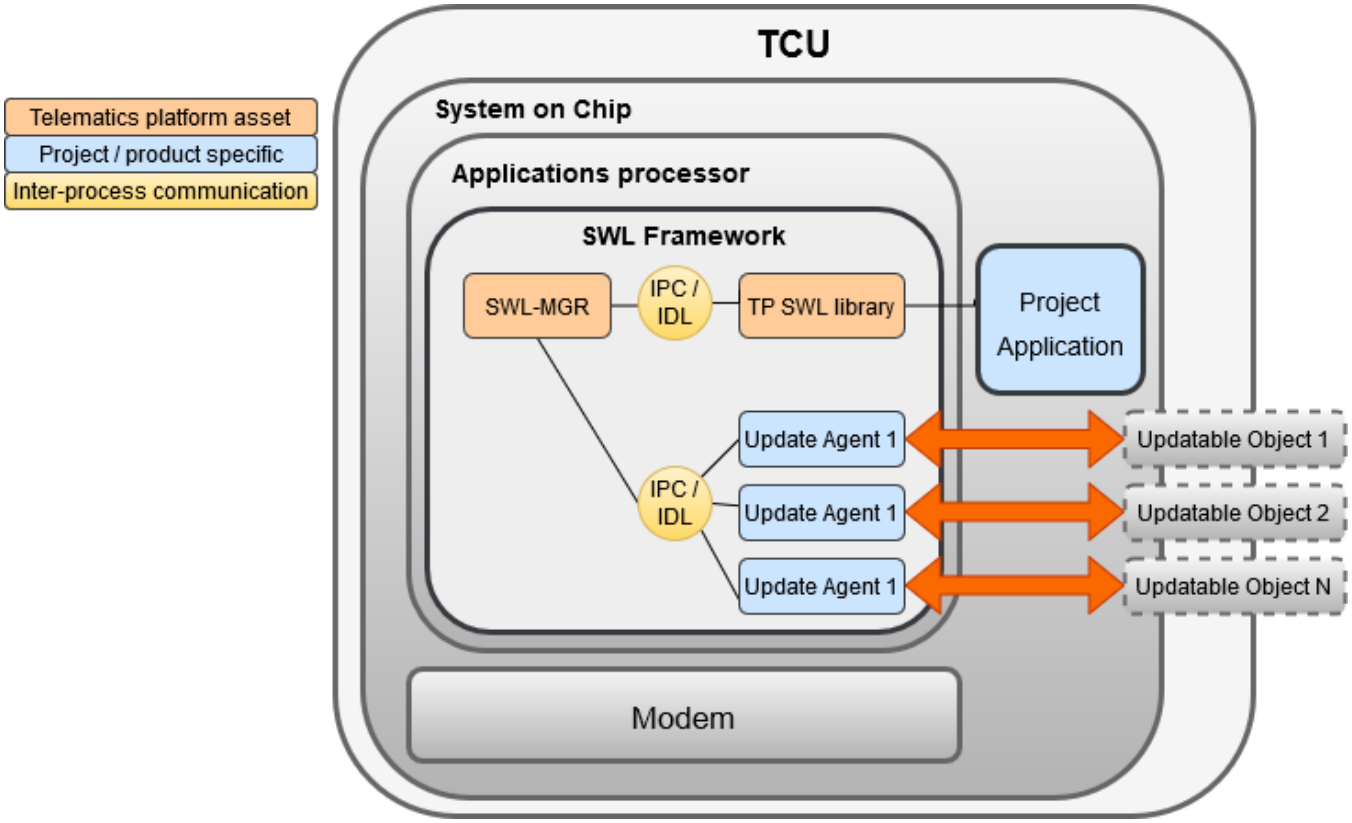

Figure 1. TP SWL Framework main elements

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 1, above shows a component diagram of some TCU elements involved on the update processes of the TCU. The sources could be a remote OTA OMA-DM server, a CAN flashing tool, or some other source; the objectives to update could be flash memory chips within the SoC or embedded firmware from other elements of the TCU or another ECU; the dotted line of the objectives try to convey that they may be in different places inside or outside the TCU. Since each project might have different sources or updatable objectives the Update Agents might vary for a given project/product.

The main goal of Software Loading Manager is to accomplish a software update using an update package that has been downloaded and validated by the Software Loading Client. The Software Loading Client obtains a validated download package and requests to perform an update. Then the Software Loading Manager with the help of the Update Agents proceeds with the actual updating of the software.

Note that Software Loading Manager is not involved in the download process. The Software Loading Client must obtain an update package (e.g., download and validate) before requesting that an update be performed using the update package.

Software Loading Manager acts as the singular point of interaction to achieve software update functionality to the Project Application.

Keep awake handling

Also, the Project Application should set the appropriate Keep-Awakes for the Software Loading Update phases where operations are in progress, which are:

- E_SWL_STATE_VAL_IN_PROGRESS
- E_SWL_STATE_UPD_IN_PROGESS
- E_SWL_STATE_ACTIVATION_IN_PROGRESS
- E_SWL_STATE_UPD_CANCEL_IN_PROGRESS
- E_SWL_STATE_UPD_ACCEPT_IN_PROGRESS
- E_SWL_STATE_UPD_REJECT_IN_PROGRESS
- E_SWL_STATE_ACTIVATION_VERIFY_IN_PROGRESS

List of Keep-Awakes to use:

- Software Loading: to avoid the Unit to go to Sleep
- Reset: to avoid reboots requested to Power Manager.
- Recovery Reset: to avoid recovery actions done by Power Manager.

For further information about keep-awake handle, refer to TP_F101U-00007.

Figure 2 Update Workflow show how the update goes through the different states during the update process.

© Continental Automotive GmbH 2019
Confidential and Proprietary

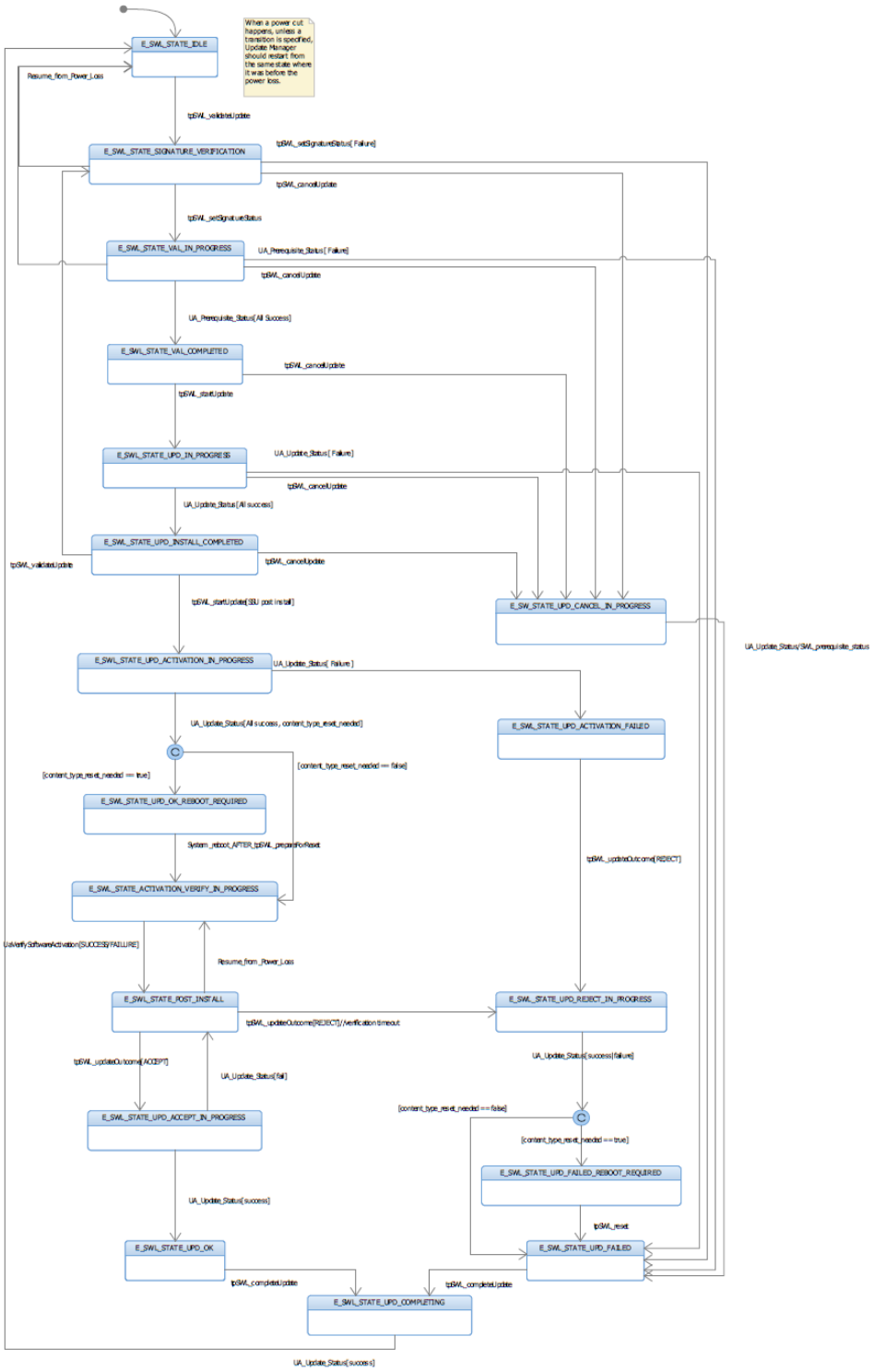

Figure 2 Update Workflow

© Continental Automotive GmbH 2019
Confidential and Proprietary

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend

- Telematics Platform
- Product Specific
- 3rd Party

2 Setup

2.1 Tools/Equipment Needed

The hardware (HW) requirements vary depending on the type of the update that wants to be tested or developed.

2.2 Setup Instructions

The details about the setup of the different hardware that can use the Software loading Manager or required for the development of a particular Update Agent is beyond the scope of this guide; however the basic testing strategy applies: the TCU elements should be programed by means of development or factory tools with the original or source content, whose images are called also referred as base or initial; a 2nd set of images referred as target, new or final, should be flashed using the Software Loading framework.

How to be a Client of Software Loading Manager

1. Use pkg-config in the client application Makefile to link to the Software Loading Manager client library

```
LDFLAGS += `pkg-config -libs swl`
CFLAGS += `pkg-config -cflags swl`
```

2. Include Software Loading Manager header files in client application source code

```
#include "software_loading.h"
#include "software_loading_types.h"
```

3. Wait for Software Loading Manager to be ready at startup.

© Continental Automotive GmbH 2019
Confidential and Proprietary

a. If client application has a “hard” dependency on Software Loading Manager to be running use system .service “After” field

```
[Unit]
After=swl.service
```

b. If client application has only a single thread (or portion of process) that is dependent on Software Loading Manager use TP service monitor API.

```
tpSVCMON_registerServiceStateChange(SWL_SERVICE,FALSE,
callback, &monitor_id);
```

4. **Security:** Usage of Software Loading Manager APIs require clients (user) to be a member of **“swl”** group.

5. Once Software Loading Manager service is ready, initialize communication with the Software Loading Manager in client application source code (**note:** pre-requisite is that the client must first call `tpSYS_initIPC()`).

```
tpSWL_Error_t swl_ret = E_SWL_ERROR_NONE;
swl_ret = tpSWL_init ();
/* check return code */
```

6. API calls "tpSWL_init" and "tpSWL_deinit" should be paired in call sequence per process. Before another “tpSWL_init” is called by a process, it has to make sure, that any possible previous “tpSWL_init” was closed by a “tpSWL_deinit”.
7. Parallel maximum processes amount that use SWL API is 40.

2.3 Setup Troubleshooting

Not applicable

3 Features/Components

3.1 Software Loading Update Package

3.1.1 Overview

3.1.1.1 Platform Content Overview

The update package is important to the update process and contains the following required data to perform an update.

a) The update package descriptor.

b) All images involved in an update.

c) All the signature files. Every file within the update package should have a corresponding signature file.

The update package is a file of ISO-9660 format usually with the extension of .iso file that gathers all the other files listed above.

The update package consists of an xml file which has the name as – update_package_desctiptor.xml. Also, this file needs to have a defined structure/arrangement of tags. As a reference the following update_package_descriptor.xml is shown:

```
<?xml version=”1.0” encoding=”UTF-8”?>
<packagecontent numberimages=”1”>
   <oem version=“OEM_TEST_VERSION“ />
   <image name=”example_image_id” filename=”example.diff” type=”multipartition”>
      <size>19176</size>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
      <newromsize>0</newromsize>
      <contenttype>content_type</contenttype>
      <type>diff</type>
      <dependentfiles>
            <file>further_structure.xml</file>
      </dependentfiles>
      <version>2.0.1</version>
      <checksum>0xdebb20e3</checksum>
      <prerequisiteversion>2.0.0</prerequisiteversion>
      <updatemode>normal</updatemode>
   </image>
   <signatures>
      <file name="example.diff" signature="example.sig"/>
      <file name="further_structure.xml" signature="further_structure.xml.sig"/>
   </signatures>
</packagecontent>
```

3.1.1.2 Product Expectations

It's expected that the user of the TP Software Loading framework is able to provide a well formed update package and its corresponding well-formed update_package_descriptor.xml with all the signature files created.

The generation of keys and signature files are responsibility of the Project Application team.

A well formed update package should have a name that complies with below list of characters:

- 'a-z': Letters lower case
- 'A-Z': Letters upper case
- '0-9': Numbers
- '-': Dash
- '_': Underscore
- '.': Dot
- '/': Slash

Update packages that contain characters that are not listed in above list will be rejected by the Software Loading Manager.

3.1.1.3 Extending Platform Functionality

Not applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not applicable

3.1.2.2 Setup Instructions

Not applicable

3.1.2.3 Setup Troubleshooting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3 Usage

3.1.3.1 Basic Instructions

The update package descriptor should be a well formed xml file and its contents should be adequate for the project, for instance an image name attribute and an image contenttype sub element should match an existing Update Agent content type otherwise the update will fail.

3.1.3.2 Detailed Instructions

The image sub elements of the update package descriptor file have several child elements and attributes of their own. Please refer to the table below for further details.

XML TAG DESCRIPTION		
TAG NAME	`<oem>`	
PARENT TAG	`<packagecontent>`	
OCCURRENCE	`1`	
DESCRIPTION	This tag holds the details about the product/OEM.	
NOTE	*It may have the name of the manufacturer/ model name etc.*	
ATTRUBUTES	version	Defines the name of the model or the manufacturer of the TCU etc.
CHILD ELEMENT	None	

XML TAG DESCRIPTION		
TAG NAME	`<image>`	
PARENT TAG	`<packagecontent>`	
OCCURRENCE	`Min=1 and Max=8`	
DESCRIPTION	This image tag holds updateable element details.	
NOTE	*Update package should have at least one element details. It can be updateable image or configuration details.*	
ATTRUBUTES	name	Defines the element identifier for the image tag. The 'name' of image tag should be unique across the file.
	Filename	Defines the element file name i.e present inside update package.
	Type	This adds an extra qualifier to the image. Basically it depicts the partitions of the target that will be flashed by the image in question.
CHILD ELEMENT	`<size>`	Defines element size.
	`<newromsize>`	Defines the expected final size after the update is applied in bytes.
	`<contenttype>`	It is an identifier that allows the Update Agent to identify which element to update.
	`<type>`	Defines type of image that helps to differentiate between full or diff updates.
	`<dependentfiles>`	Defines the dependent files for this element. Dependent files should be present inside update package. The file names are provided in <file> tag as child elements of this tag.

XML TAG DESCRIPTION		
TAG NAME	`<signatures>`	
PARENT TAG	`<packagecontent>`	
OCCURRENCE	`1`	
DESCRIPTION	This signature tag holds the signature files for all updatable element.	
NOTE	*Update package can have the signatures for the element.*	
ATTRUBUTES	None	
CHILD ELEMENT	file	Defines signature file details for each image element

XML TAG DESCRIPTION	
TAG NAME	`<file>`
PARENT TAG	`<signatures>`

© Continental Automotive GmbH 2019
Confidential and Proprietary

OCCURRENCE	The same number of times the image tag appears.	
DESCRIPTION	This tag holds signature details of the updateable element details.	
NOTE	*It should be the same as the number of image elements.*	
ATTRUBUTES	name	Defines the name of the file for which signature is created. Same as the value put in filename
	signature	Defines the name of the signature file for the corresponding element. The name should be same as the filename tag but suffixed by a ".sig"
CHILD ELEMENT	None	

Table 1: Content details of update_package_descriptor.xml

3.1.3.3 Testing/Error Reporting

Not applicable

3.2 Software Loading Persistency

3.2.1 Overview

3.2.1.1 Platform Content Overview

Software Loading persistency refers to the capability of the system to allow Project Application (also other modules) to store certain values in the non-volatile memory such that the system can avail these values persistently without maintaining them individually. The Project Application needs to be a part of the "swl" group to access the keys.

3.2.1.2 Product Expectations

Since down times are expected to happen during the update process, Project Application is responsible of ensuring the system is aware of the update process. For example, system configuration should not try to reboot the board if communication link is down during the update process up to Post-Install phase.

Project Application needs to set the value of the swl_verification_timeout key before an update is executed. To access these keys the project application needs to be part of the "swl" group.

3.2.1.3 Extending Platform Functionality

Not applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not applicable

3.2.2.2 Setup Instructions

Not applicable

3.2.2.3 Setup Troubleshooting

Not applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

The keys and a brief description about their significance is provided in the Table 2.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.3.2 Detailed Instructions

The following keys are to be set by Project Application.

Keys	Description	Default values
swl_verification_timeout	Persistence key to set Verification timeout.	300 seconds
MANUFACTURER	Persistence key to set the manufacturing unit's name for the PD. It is of string type.	
MODEL	Persistence key to set the name of the model of which the TCU becomes a part. It is of string type.	
FW_ID	Persistence key to set the ID for the firmware. It is of string type.	
swl_internal_verification_timeout	Persistence key to set internal verification timeout in seconds. It is the maximum time the Software Loading Manager will wait for the Project Application to approve or reject the Update. It is activated if its value greater than 0, current update state is E_SWL_STATE_POST_INSTALL and the boot operation mode is SSU. To disable it, set its value to 0.	0 seconds
swl_update_timeout	Persistence key to set the timeout value for the Dbus call. The value must be initialized from Product Application side previous any interaction with Software Loading Manager. **Note:** Persistence timeout value can be overwritten by Update Agent custom value if the Update Agent has the capability for it.	300 seconds
swl_allow_same_version_write	Persistence key to set the expected behavior from the update agents in case of an update of the same content type version is requested. If enabled, the update agent is expected to exercise the updating process of the image if supported. If disabled there is no expectation defined. To enable it, set its value to 1.	0 (disabled)
swl_max_retries_ua_complete_update	Persistence key to set the maximum number of complete update retries to perform on the corresponding update agent that failed during the complete update process (either with or without integrity verification). If retries are exhausted Software Loading Manager will return an error to Project Application. Changing the persistence key value will enable the retry mechanism and implies that it will increase the time taken to complete the update process.	0 retries

© Continental Automotive GmbH 2019
Confidential and Proprietary

Keys	Description	Default values
swl_complete_update_verification_check_enable	Persistence key to set CRC verification to verify the integrity of the new updated images after the complete update process. In case that this key disabled, or if the update agent doesn't suppot complete update verification capabilities, the complete update will be performed without integrity verification. The user must consider that when this key is enabled, the time to complete the software update process will increase in comparison with the complete update without CRC verification.	1 (enabled)
swl_max_rollback_retries	Persistence key to set the maximum number of retries the software will attempt to perform the rollback process after a rollback failure. Modifying the persistence key value will enable the retry mechanism and implies that it will increase the time taken to complete the update process.	0 retries

Table 2: Persistency keys to set by PD

3.2.3.3 Testing/Error Reporting

Not applicable

3.3 Software Loading Validation

3.3.1 Overview

3.3.1.1 Platform Content Overview

The Project Application may perform some checks on the update package per the requirements for the update package wrappings. When ready the Project Application can start the validation.

The validation phase covers the integrity of the update package, the verification of the update_package_descriptor.xml file; verification of signature file(s), sufficient non-volatile storage space available for update process from Project Application and finally the prerequisites check or dry-run of the Update Agents over their corresponding image files. Platform supports update of multiple update packages from Product Application.

3.3.1.2 Product Expectations

In case some wrapping over the TP update package are in place per project specifications the Project Application should take care of extracting the package.

The key storage and signature file creation and verification is responsibility of the Project Application team.

The contents of the package can be verified for integrity, security and other project defined checks and is integral to the update process. This needs to be correctly accomplished by Project Application.

The Project Application should ensure that there is sufficient non-volatile storage space available for the software update process. The Project Application can free up non-essential non-volatile memory to support the update (e.g., log file storage).

The Project Application can query the update status to ensure that an update process can be started.

© Continental Automotive GmbH 2019
Confidential and Proprietary

The Project Application should set the the appropriate Keep-Awakes as documented in the keep awake handling sub-section before starting Validation operation.

3.3.1.3 Extending Platform Functionality

Not applicable

3.3.2 Setup

3.3.2.1 Tools/Equipment Needed

Not applicable

3.3.2.2 Setup Instructions

Not applicable

3.3.2.3 Setup Troubleshooting

Not applicable

3.3.3 Usage

3.3.3.1 Basic Instructions

In order to perform the validation over a TP update package the following APIs should be used:

API/Type Description	Doxygen Path
Register to receive notification that check signature is required.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerSignatureFoundCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterSignatureFoundCallback**
Register to receive update status.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerUpdateStatusCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterUpdateStatusCallback**
Allows a client application to request to validate an update.	Modules |-> *Software Loading* |-> *Software Loading Manager (SWL)* |-> **tpSWL_validateUpdate**
The client application sends status to the server.	Modules |-> Software Loading

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	Doxygen Path
	|-> Software Loading Manager (SWL) |-> tpSWL_setServerStatus
The client application on request to perform a signature check uses this method to return the result to the platform.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> tpSWL_setSignatureStatus

3.3.3.2 Detailed Instructions

Figure 3 Validation of SWL highlights the steps in update validation process and the possible outcomes.

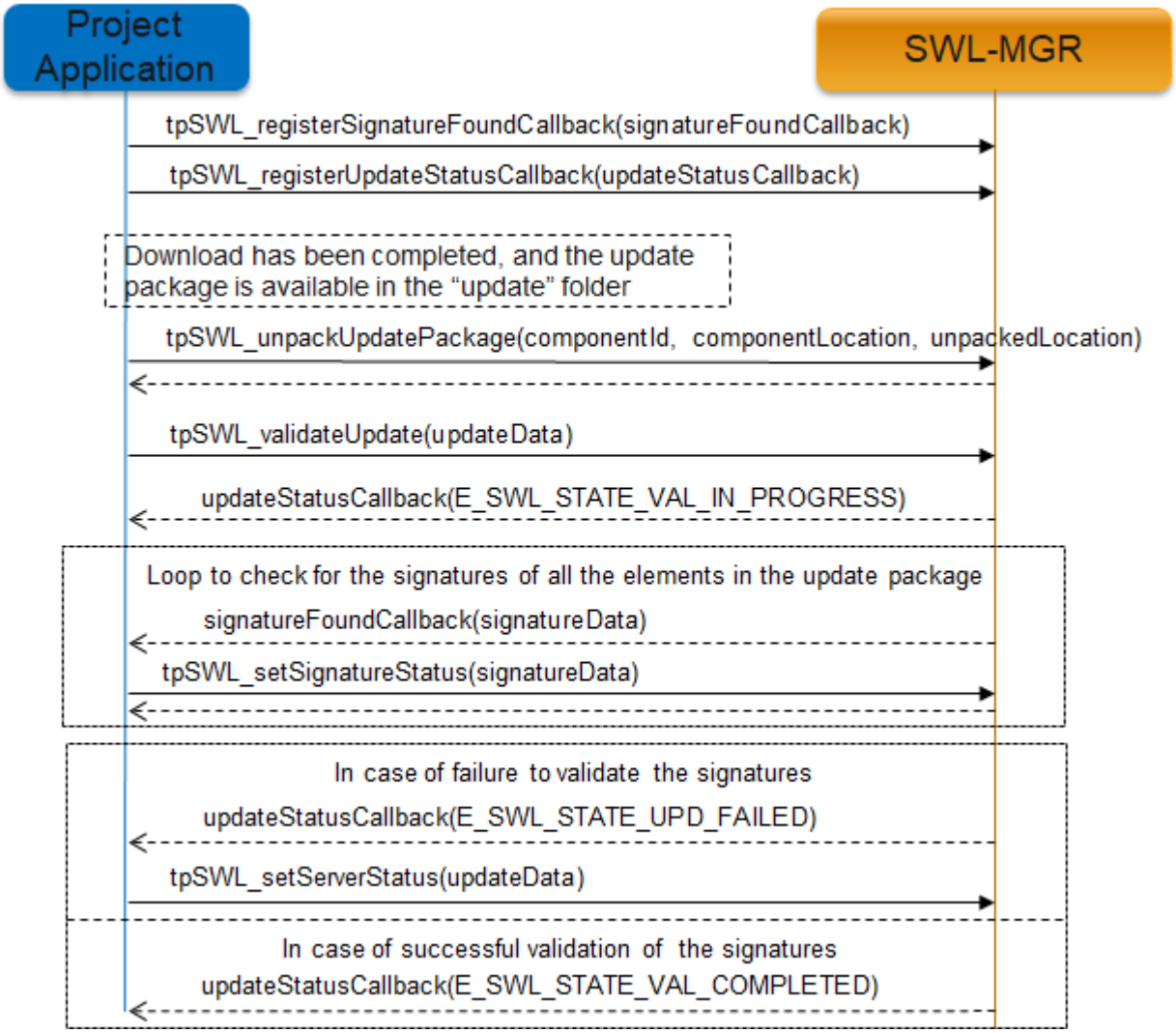

Figure 3 Validation of SWL

Note: The cleanup process will be reattempted for a failed cleanup during the previous update process (see E_SWL_STATE_COMPLETING_UPD in Post-Install phase in section 3.4.3.2). Validation process may be extended due the cleanup process reattempt.

© Continental Automotive GmbH 2019
Confidential and Proprietary

As a feature to support multiple update packages at a time, Software Loading will allow the validation of further update packages provided by the Product Application, even when an update package has been installed (ie. The Software Loading state is “E_SWL_STATE_UPD_INSTALL_COMPLETED”)

3.3.3.3 Testing/Error Reporting

Not applicable

3.4 Software Loading Update

3.4.1 Overview

3.4.1.1 Platform Content Overview

The update phase covers the application of the update package to the SoC. Usually, the update phase consists of individual application of diff or full images as per the updatable elements found in the update_package_descriptor.xml file.

There may be additional verification done before the package is actually applied.

The update phase allows seamless updates and has rollback capabilities. Associated with E_SWL_UPGRADE_MODE_SSU upgrade mode. The update is prepared during install phase and services will be running in normal operation. The new images are activated, and services will be down during post-install phase. Project Application can trigger rollback operations in case the update is not approved or fails. In case the rollback operation fails, the system can be considered as left in an inconsistent state from which the software can’t recover.

Once the update package is correctly applied an UpdateStatus notification will be emitted, at this point with state E_SWL_STATE_UPD_OK_REBOOT_REQUIRED to the Project Application. In case of any error in the update process the state may also be E_SWL_STATE_UPD_FAILED_REBOOT_REQUIRED. This signifies that the SoC has completed applying the update package (may be successfully or unsuccessfully) and a cleanup of the update folder is needed. For this the `tpSWL_startReset` API is to be called by the Project Application to have controlled reboots during the update process.

Platform supports update of multiple update packages from Product Application at the same time.

There is functionality to free space from the SoC non volatile memory within update process based on /etc/resmgr.cfg contents. The contents of the file is just a list of paths that can be removed while the update process is running. Whenever the update in progress (status different than E_SWL_STATE_IDLE) files listed there will be continuously removed. Once the update is finished (status is E_SWL_STATE_IDLE) the removal process will be stopped. Also, whenever the amount of space is less than 1MB, files listed in /etc/resmgr.cfg will be removed. The paths in /etc/resmgr.cfg are fully qualified and wildcards are allowed.

3.4.1.2 Product Expectations

1. The Project Application should set the the appropriate Keep-Awakes as documented in the keep awake handling sub-section before starting the Update operation.
2. Project Application may complete all long running processes like UI requests before calling the `tp_SWL_startUpdate` API.
3. Project Application must register a callback for update status using `tpSWL_registerUpdateStatusCallback` API in order to process status and failReason parameters as defined by tpSWL_UpdateStatusData_t.
4. Project Application needs to unpack the update package using the `tpSWL_unpackUpdatePackage()` API and read the contents of update_package_descriptor.xml to extract the upgrade_mode from it, which will need to be sent as a parameter to the `tpSWL_startUpdate` API.
5. The client application should register to the callback `tpSWL_SoftwareActivationVerificationResultCallback_t` using

© Continental Automotive GmbH 2019
Confidential and Proprietary

`tpSWL_registerSoftwareActivationVerificationResultCallback` API to receive verify software activation result emitted by the Software Loading Manager. The client application on receiving the verify software activation result should handle it according to the possible update states:

a. UPD_FAILED_REBOOT_REQUIRED: At this point `tpSWL_updateOutcome` was already invoked, so:
   - If client application handles the software activation they need to retry their operations.
   - If client application does not handle the software activation, it could trigger another reboot to retry the operation.

b. UPD_FAILED: Client application should propagate the error to indicate that the software might not be in a stable state. `tpSWL_completeUpdate` can still be invoked to cleanup the current update process and trigger a new one.

6. The Software Loading Manager waits for the outcome of the update, which the Project Application can send using the `tpSWL_updateOutcome()` API on receiving the verifyUpdate callback. If the Software Loading Manager does not receive the update outcome (in time) or the update is rejected, it rolls back to the present software before the update.
7. Project Application needs to call `tpSWL_prepareForReset` API so preparations can be made before requesting a board reset.
8. Project Application can register a callback for prepareForReset status using `tpSWL_registerPrepareForResetStatusCallback` API in order to process status from prepareForReset operation asynchronously.

9. PrepareForReset signal should be received before start_reset signal or else an error will be reported back to start _reset and the reset request will fail
10. Project Application needs to call *`tpSWL_startReset`* API for having controlled reboot of the system.
11. Project Application must also call *`tpSWL_completeUpdate`* API to complete the update process by doing a final cleanup after the target boots up from the above reset.
12. Software Loading Manager allows a single execution (for all update packages) of Activation phase, update outcome execution and complete update execution at a time.
13. Project Application optionally may consider using `/etc/resmgr.cfg` file to help free up space for Software Loading Manager if needed.

Note:

If no callback is registered then `tpSWL_prepareForReset` will be treated synchronously. In this case prepareForReset operation will be completed when returning the control to the Project Application.

If a callback is registered, prepareForReset operation will be treated asynchronously, in this case the operation is completed once the callback is invoked. The time between prepareForReset completion (either asynchronously or synchronously) and `tpSWL_startReset` should be minimal to ensure that the reset operation is done correctly. If prepareForReset signal is not received from Product application, then a failure is returned with fail reason "E_SWL_FAIL_PREPARE_FOR_RESET_SKIPPED".

3.4.1.3 Extending Platform Functionality

Project Application may keep a track of the progress of the update by registering for a callback using the `tpSWL_registerUpdateProgressCallback` API.

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.4.2.2 Setup Instructions

Not applicable

3.4.2.3 Setup Troubleshooting

Not applicable

3.4.3 Usage

3.4.3.1 Basic Instructions

In order to perform the update with a TP update package the following APIs should be used:

API/Type Description	Doxygen Path
Register to receive regular update progress.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerUpdateProgressCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterUpdateProgressCallback**
Register to receive update status.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerUpdateStatusCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterUpdateStatusCallback**
Allows mounting an ISO image on a location based on the component ID provided by the caller.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_unpackUpdatePackage**
Allow a client application to request to begin an update.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_startUpdate**
Register to receive proceed start update.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerProceedStartUpdateCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterProceedStartUpdateCallback**

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	Doxygen Path
Register to receive requests to install SCOMO components.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerInstallComponentRequestCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterInstallComponentRequestCallback**
Register to receive verify update.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerVerifyUpdateCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterVerifyUpdateCallback**
Register to receive an indication that platform needs to reset the board.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerResetNeededCallback**
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterResetNeededCallback**
Allow a client application to notify the result of the validation.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_updateOutcome**
Notify Software Loading Manager to prepare for board reset.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_prepareForReset**
Register to receive prepareForReset status.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerPrepareForResetStatusCallback**

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	Doxygen Path
Deregister a previously registered callback.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterPrepareForResetStatusCallback**
The client application can request a reset of the board. After the update the client application can request the reset.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_startReset**
Allow a client application to request to complete update.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_completeUpdate**
This structure defines the update status data.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_UpdataStatusData_t**
Notify Software Loading Manager that client is ready to receive events.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_clientStarted**
Notify Client application the software activation verification result	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_SoftwareActivationVerificationResultCallback_t**
This enumeration defines the software activation result	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_SoftwareActivationResult_t**
Register to receive Software Activation Verification Result	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerSoftwareActivationVerificationResultCallback**
Deregister to receive Software Activation Verification Result	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterSoftwareActivationVerificationResultCallback**

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.4.3.2 Detailed Instructions

The project application can register to receive the update status using the `tpSWL_registerUpdateStatusCallback` API. Once the package is validated completely by the Project Application an UpdateStatus notification will be emitted, at this point with state E_SWL_STATE_VAL_COMPLETED to the Project Application. This notification indicates that the update package is consistent in terms of signature, integrity and authenticity, hereafter the Project Application needs to trigger the software update. The Software update phase covers two phases of the update process; install and post-install. Which are described below in more detail:

1. Install phase: During this phase the current running images are only prepared for the update and should be left unchanged. For instance: an Update Agent may create copies of the images linked to its content type and apply the diff or full images on those copies. In case of failure, the update process can be aborted while the original images are still operating since they haven't been modified. Another Update Agent may do setup operations for the actual update. Also, note that services should be operational and unaffected by the Update operation. The states related to this phase are:
   a. E_SWL_STATE_UPD_IN_PROGRESS: The image update preparation is in progress, during this state the Software Loading Manager parses the update_package_descriptor.xml to extract the images to update and call each Update Agent related to them.
   b. E_SWL_STATE_UPD_INSTALL_COMPLETED: The image update preparation is finished.
2. Post-Install phase: Also called activation phase, this is where the actual update happens. The currently running images should be replaced with the new ones. Depending on the images that are being updated a reboot may be required or not. During this phase services may be down. The states related to this phase are:
   a. E_SWL_STATE_UPD_ACTIVATION_IN_PROGRESS: Each image is being replaced with the new ones. Update Agents take care of this process by themselves and report success or failure of this operation. In case one Update Agent reports failure for the activation of its corresponding image, an automatic rollback operation will be performed by the Software Loading Manager. Also, an automatic reboot could be triggered at this point to allow the rollback to proceed correctly. Once the rollback is finished, the next state will be E_SWL_STATE_UPD_FAILED_REBOOT_REQUIRED.
      1. In order to transition to the next state a controlled reboot should be triggered by calling `tpSWL_prepareForReset()` API.
   b. E_SWL_STATE_ACTIVATION_VERIFY_IN_PROGRESS: Once a controlled reboot is triggered this transition is entered to indicate that Sofware Loading Manager is querying each Update Agent involved in the update process to verify that the software was properly activated. The verification involves accessing read-only resources with information about the activation status. Project Application should suspend Software Loading activities until this verification is completed. The purpose is to ensure that the verification is completed and the result is properly reported. Two outcomes will be reported once this operations is completed:
      1. E_SWL_STATE_POST_INSTALL with update status as success.
      2. E_SWL_STATE_POST_INSTALL with update status as failed.
   c. E_SWL_STATE_POST_INSTALL: The image activation process is finished. At this point update status could be success or fail, based on that result and other additional checks from Project Application, update could be rejected or accepted using `tpSWL_updateOutcome()` API. Since `tpSWL_updateOutcome()` is asynchronous operation, it is called in a way that doesn't timeout the system or Dbus from Product Application code and it's up to Product Application to decide how to ensure timeout events don't happen from its side. By default the **timeout** is set to 25 seconds; this **can be changed using swl_update_timeout persistency key, but** to make use of it follow user guide – TP_103U-00026_User_Guide_Persistence. Upon receipt of verifyUpdate in this state, project application needs to obtain update status using `tpSWL_getUpdateStatus()`**.** Depending on retrieved status the following behavior expected:
      1. If status = FAIL, call `tpSWL_updateOutcome()` with E_SWL_RESULT_REJECT.
      2. If status = SUCCESS, the project application should perform actions indicated for "after the reboot stage in post install state" below.
   d. E_SWL_STATE_UPD_ACCEPT_IN_PROGRESS**:** This state is added to indicate that the update_outcome has been accepted and switch to the updated set of images.

© Continental Automotive GmbH 2019
Confidential and Proprietary

e. E_SWL_STATE_UPD_REJECT_IN_PROGRESS**:** This state is added to indicate that update_outcome has been declined and will be followed by the rollback mechanism.
f. E_SWL_STATE_UPD_ACTIVATION_FAILED**:** This state is added to indicate failure to activate new set of images from update agent side.
g. E_SWL_STATE_COMPLETING_UPD**:** During this phase irrespective of if the tpSWL_updateOutcome() "ACCEPTED" or "REJECTED", it cleans up all the temporary files created during the update process. After completion of this phase software loading enters into E_SWL_STATE_IDLE state. In case of a failure on the cleanup process, a retry will be performed during the next update cycle in the Validation phase. An error will be reported in any of the cleanup failures.

Once the update process is finished, Software Loading Manager will notify that a reset is needed and the Project Application will receive this notification through the callback registered by tpSWL_registerResetNeededCallback API. When the `tpSWL_startReset()` API is called, the update state is changed to E_SWL_STATE_POST_INSTALL. During this state, Update Agents may perform post-install operations that could marked the update as failed. This is done in two different stages.

1. Before the reboot: Update Agents may perform post-install operations which could fail and as such, they are reported to the Software Loading Manager which propagates the error to Project Application by using the UpdateStatus event. Since this operation is happening during a reboot operation, the Project Application is responsible of verifying that the Update Status is still E_SWL_STATUS_SUCCESS once the reboot operation is finished and system is running with the new images.
2. After the reboot: Project Application will receive verifyUpdate notification to perform the required checks to confirm that the Software functionality works according to the Project Application needs (such as eCall, data call, etc.). After Project Application completes this verification, it is expected to call `tpSWL_updateOutcome()` API to mark an update as accepted. At this stage, Update Agents may perform post-install operations which could fail and as such, they are reported to the Software Loading Manager which propagates the error to Project Application by replying a value different to E_SWL_ERROR_NONE to the `tpSWL_updateOutcome()` call.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 4 Update setup shows the required setup steps for the update process before entering the install phase.

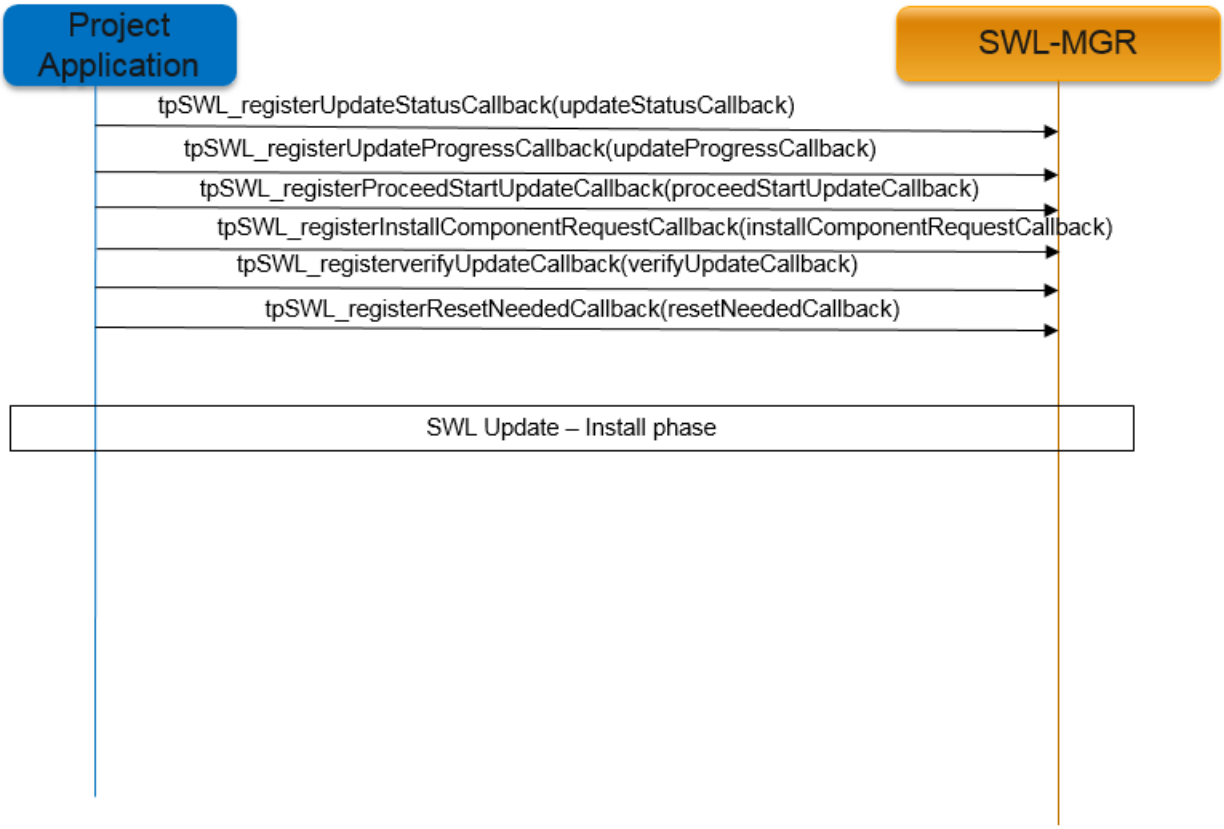

Figure 4 Update setup

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 5 Update install phase shows the update process in its install phase once the setup process is finished

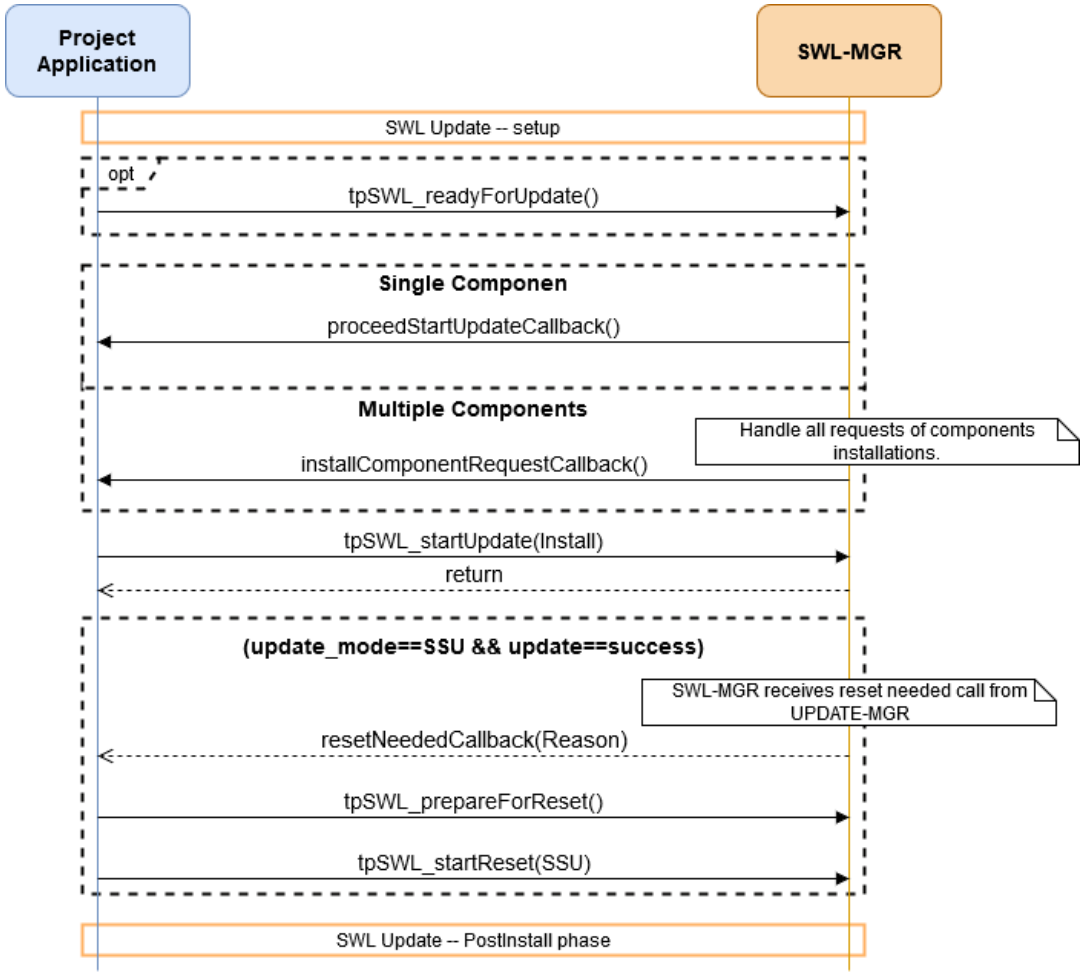

Figure 5 Update install phase

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 6 Update post-install phase shows the update process in its post-install phase after install phase has been completed successfully.

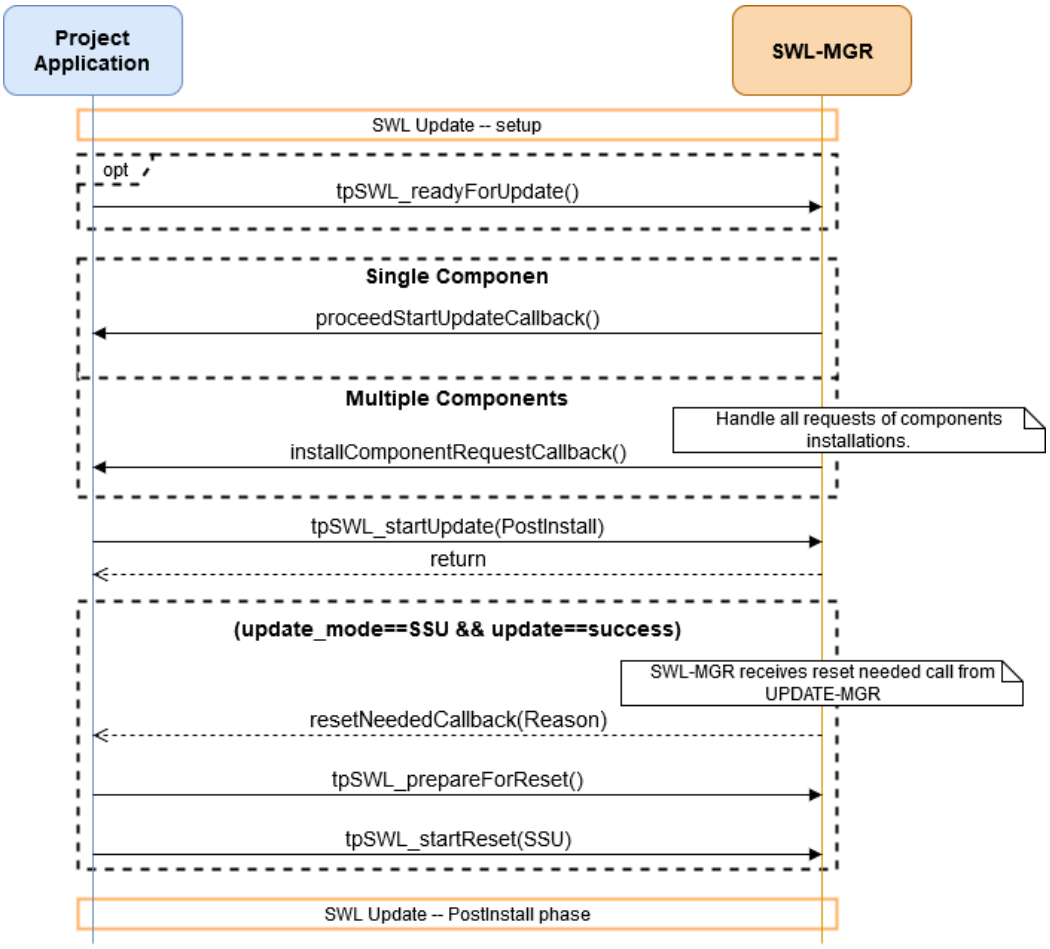

Figure 6 Update post-install phase

© Continental Automotive GmbH 2019
Confidential and Proprietary

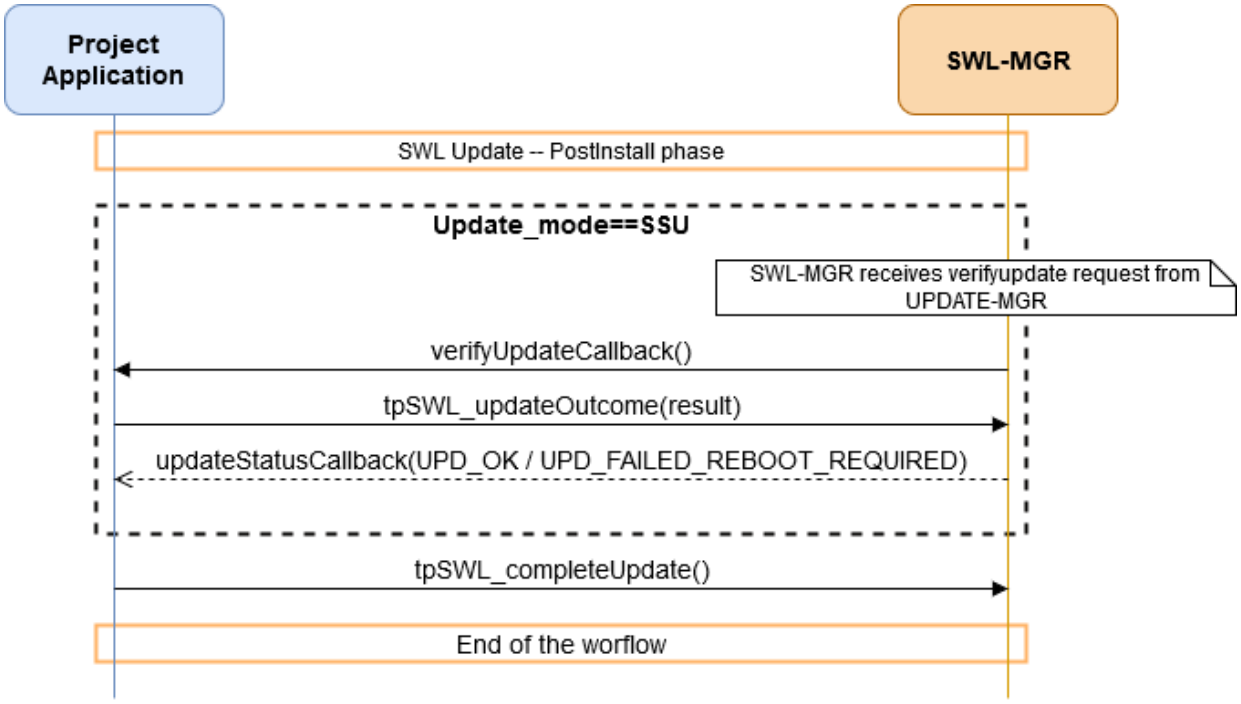

Figure 7 Booting with new Software shows the final step of the update process in a sunny day scenario.

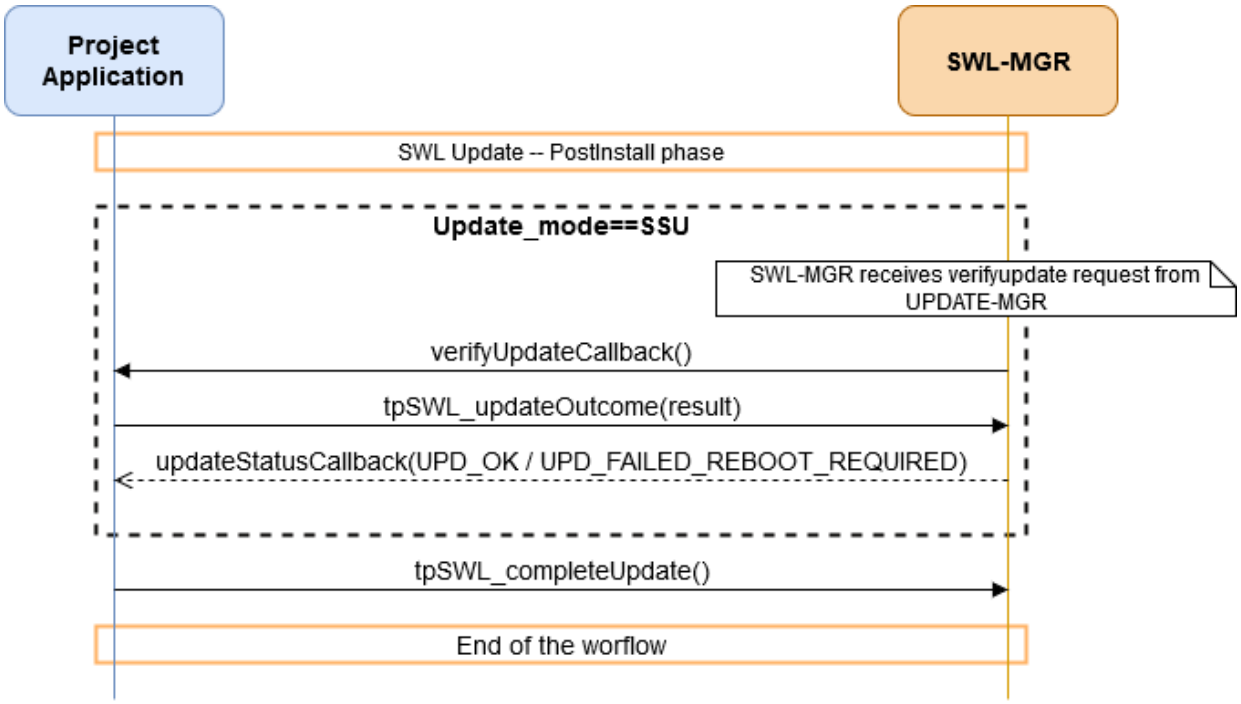

Figure 7 Booting with new Software

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.4.3.3 Update workflows

The update process supports different workflows. The workflow is determined by the upgrade_mode parameter from the `tpSWL_startUpdate` API. The values associated with upgrade_mode parameter are defined by tpSWL_UpgradeMode_t.

3.4.3.4 Testing/Error Reporting

Not applicable

3.5 Software Loading Cancel

3.5.1 Overview

3.5.1.1 Platform Content Overview

The cancel operation covers the capability of aborting an already running update. This feature is only supported for SSU before post-install phase. Once the update process has reached the post-install phase, the cancel operation is no longer possible, alternatively the Project Application can use `tpSWL_updateOutcome()` with E_SWL_RESULT_REJECT.

3.5.1.2 Product Expectations

1. The Project Application should set the the appropriate Keep-Awakes as documented in the keep awake handling sub-section before starting cancel operations.
2. Project Application can trigger cancel operation using `tpSWL_cancelUpdate` API only when the Software Loading Manager state allows the operation, otherwise E_SWL_ERROR_INVALID_STATE will be returned.
3. Project Application must also call the `tpSWL_completeUpdate()` API to complete the cancel process by doing a final cleanup.

3.5.1.3 Extending Platform Functionality

Not applicable

3.5.2 Setup

3.5.2.1 Tools/Equipment Needed

Not applicable

3.5.2.2 Setup Instructions

Not applicable

3.5.2.3 Setup Troubleshooting

Not applicable

3.5.3 Usage

3.5.3.1 Basic Instructions

In order to perform the cancel with a TP update package the following APIs should be used:

API/Type Description	Doxygen Path

© Continental Automotive GmbH 2019
Confidential and Proprietary

Allow a client application to request to cancel an update.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_cancelUpdate**

3.5.3.2 Detailed Instructions

When cancel operation is triggered, Software Loading Manager will change its state to E_SWL_STATE_UPD_CANCEL_IN_PROGRESS, in order to finish any pending activity being executed by the Software Loading Manager itself or Update Agents. Once those pending activities are completed, the state will be changed to E_SWL_STATE_UPD_FAILED.

Project Application can trigger cancel operation during below states:

1. E_SWL_STATE_SIGNATURE_VERIFICATION: During this state the signature verification process is being carried. Project Application can trigger the cancel operation at this point and the Software Loading Manager will cancel the update operation without waiting for other activity to be completed.
2. E_SWL_STATE_VAL_IN_PROGRESS: During this state the image validation process is being carried. If Project Application triggers the cancel operation when the Software Loading Manager is in the middle of an image verification request, the Software Loading Manager will wait until such process is finished. Once the image validation is completed, the Software Loading Manager will stop requesting more images to validate.
3. E_SWL_STATE_VAL_COMPLETED: During this state the image validation is completed, the Software Loading Manager is waiting for the notification to start install phase. Project Application can trigger the cancel operation at this point and the Software Loading Manager will cancel the update operation without waiting for other activity to be completed.
4. E_SWL_STATE_UPD_IN_PROGRESS: During this state the update is in progress for install phase. Since the currently running images are not being modified, it is possible to cancel the operation at this point. If Project Application triggers the cancel operation when the Software Loading Manager is in the middle of an image update, the Software Loading Manager will wait until such process is finished. Once the image update is completed, the Software Loading Manager will stop requesting more images to update.
5. E_SWL_STATE_UPD_INSTALL_COMPLETED: The image update preparation is finished. Project Application can cancel the update process at this point.

Project Application must register a callback for update status using `tpSWL_registerUpdateStatusCallback` API in order to process status and failReason parameters as defined by tpSWL_UpdateStatusData_t.

- In case cancel operation succeeded, state is set to E_SWL_STATE_UPD_FAILED, status is set to E_SWL_STATUS_CANCELLED and failReason is set to E_SWL_FAIL_NONE.
- In case cancel operation failed, state is set to E_SWL_STATE_UPD_FAILED, status is set to E_SWL_STATUS_CANCELLED and failReason is set to E_SWL_FAIL_UPDATE_CANCEL_FAILED.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Error! Reference source not found. shows the cancel operation process for the valid states.

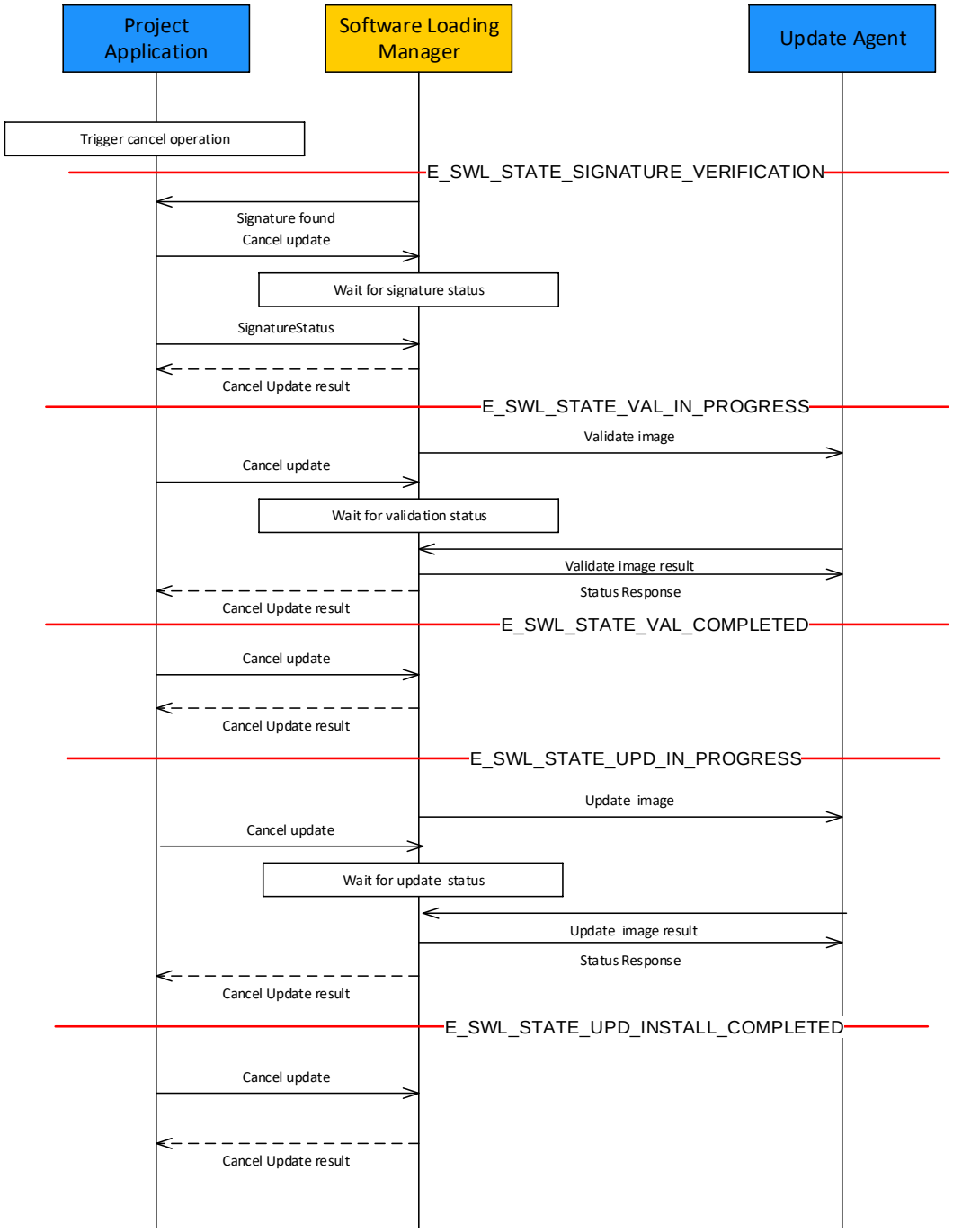

Figure 8 Cancel Update

3.5.3.3 Testing/Error Reporting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6 Software Loading Information Retrieval

3.6.1 Overview

Software Loading provides capabilities to retrieve information from the verifiable components.

A verifiable component is a piece of software running on the target and can be queried to retrieve CRC and version information to verify its contents.

Information retrival is provided to comply with the UNECE guidelines.

3.6.1.1 Platform content Overview

The Project Application can use the `tpSWL_getImageData()` and `tpSWL_verifyImages()` APIs to extract CRC and version information from verifiable components that are currently running on the target. There is a target script: “swl_verify_images_version_crc” which automates the execution of the `tpSWL_verifyImages()` API for each of the supported components.

Also the “image_crc_versions.csv” file is provided as a release artefact and will contain CRC and version information of all the components.

Notes:

1. The “image_crc_versions.csv” file contains multiple entries for the SBL component. One per each of the HW variants.
2. The “swl_verify_images_version_crc” is located in the test partition, so, **if this partition is not flashed, the script won’t be available.**

3.6.1.2 Product Expectations

The Project Application can get the list of names of all the verifiable components via the API `tpSWL_getImageData()`. To get information regarding the CRC and version, `tpSWL_verifyImages()` API or “swl_verify_images_version_crc” can be used; and this information can be compared with the one provided by the “image_crc_versions.csv” file.

It is expected that with the CRC and version data provided, the product application can perform necessary verification as per UNECE guidelines.

3.6.1.3 Extending Platform Functionality

Not Applicable

3.6.2 Setup

3.6.2.1 Tools/Equipment Needed

Not Applicable

3.6.2.2 Setup Instruction

Not Applicable

3.6.2.3 Setup Troubleshooting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.3 Usage

3.6.3.1 Basic Instructions

In order to retrieve the component details like CRC and version from the installed components in the target the following APIs should be used:

API/Type Description	Doxygen Path
Fetch the names of all the verifiable components.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_getImageData**
Fetch the CRC and version data of each verifiable component.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_verifyImages**
Register to receive CRC and version of reported image name from Update Agent.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_registerVerifyImageResultCallback**
Deregister to receive CRC and version related to component data for verification.	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_deregisterVerifyImageResultCallback**
Structure to store the component type and name	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_ImageData_t**

3.6.3.2 Detailed Instructions

As a part of the image verification process, consider the following items:

- Image version size is 32 characters.
- Image name is a string with maximum size of 128 characters.
- The format of the “image_crc_versions.csv” file is: <Image Name>,<Image Type>,<Image CRC>,<Image Version>
- Product application need to `use tpSWL_getImageData`() API to obtain the list of verifiable components available for any Update Agent. This list is used to specify image names when requesting CRC and version information via `tpSWL_verifyImages()` API.
- A deterministic list of verifiable components will be provided by `tpSWL_getImageData()` API (e.g., always in same order). The list is provided via structure `tpSWL_imageData_t`.
- `tpSWL_verifyImages()` API response for program image type (i.e., type is E_SWL_PROGRAM_IMAGE_TYPE) contains a CRC and version.
- `tpSWL_verifyImages()` API response for configuration data image type (i.e., type is E_SWL_CONFIGURATION_IMAGE_TYPE) contains a CRC only, version will be set to empty string.

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Software Loading Manager does not perform CRC or version verification. The comparison of the information retrieved by the `tpSWL_verifyImages()` API and the "image_crc_versions.csv" information is the responsibility of the product application as appropriate.

3.6.3.3 Testing/Error Reporting

Not Applicable

4 Known Issues

The Software Loading Manager only serves a single Project Application one whole update operation at a time.
Dependency on the 3rd party OMA-DM solution for the OTA updates flow.

5 FAQs

Not applicable

6 Troubleshooting/Tips and Tricks

Covered in previous section **Error! Reference source not found.**.

7 References

7.1 Abbreviations/Acronyms/Definitions

Not applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00007: Power Manager and Lifecycle Manager User Guide
TP_F101U-00026: Persistence User Guide

7.3 Website links

- Dbus information: https://www.freedesktop.org/wiki/Software/dbus/
- UNECE specification: https://unece.org/fileadmin/DAM/trans/doc/2020/wp29/ECE-TRANS-WP29-2020-080e.pdf

© Continental Automotive GmbH 2019
Confidential and Proprietary

8 Revision History

Version	Date	Description
01.00	Aug 14, 2019	Initial release
02.00	Aug 29, 2019	3.2.1.2 Application is responsible of notifying the system about the update process 3.2.3.2 Removed references to VuC persistence variables
03.00	Dec 3, 2019	Removed all references of “Recovery mode”. Added tpSWL_prepareForReset() referemces Updated Figures 5 and 6
04.00	Mar 22, 2020	Updated persistence key table with “swl_update_timeout” Added complete update state Added fail reason for “prepare for reset”
05.00	Jun 9, 2020	Removed deprecated information Updated persistence keys default values Updated Figures Values Added Figure 8 to see the update workflow
06.00	Jul 6, 2020	Replace tpSYS service notify API with tpSVCMON API. Software Activation Verification
07.00	Aug 12, 2020	Update Product expectations for Validation process with information related to available space.
08.00	Aug 27, 2020	Details to query update status.
09.00	Sep 8, 2020	Format changes to table information.
10.00	Nov 23, 2020	Verify update handling during post install
11.00	Dec 15, 2020	Free space from data within update process
12.00	Jan 11, 2021	Cancel Update new transitions.
13.00	Mar 2, 2021	Software Loading Information Retrieval
14.00	Apr 8, 2021	Software Loading Information Retrieval extra details
15.00	Jul 9, 2021	Software Loading prepareForReset asynchronous behavior
16.00	Jul 9, 2021	Usage of swl_allow_same_version_write persistence key
17.00	Sep 30, 2021	Add description of swl_max_retries_ua_complete_update and swl_complete_update_verification_check_enable persistence keys. Add persistence key description swl_max_rollback_retries description and SoftwareActivationVerificationResult details. Add note on persistence key about Update Agent custom timeout.
18.00	Nov 8, 2021	References to KeepAwake
19.00	Jun 10, 2024	tpSWL_init/tpSWL_deinit sequence rule per process

© Continental Automotive GmbH 2019
Confidential and Proprietary